PHP Feature & Vulnerability Detection System

Agent-Ready Software Requirements Specification & Implementation Blueprint

DOCUMENT VERSION	PRIMARY STACK	TARGET ARCHITECTURE	TARGET AUDIENCE
2.0 (Production Ready)	**Python / FastAPI / React**	**Modular Evidence-Driven**	**AI Coding Agents & DevSecOps**

1. EXECUTIVE SUMMARY

The **PHP Feature and Vulnerability Detection System** is an evidence-driven web application security assessment platform. It accepts a target URL, examines publicly accessible website information, identifies indicators of PHP technology, discovers application features, executes configured security checks, and generates structured assessment reports.

> **Core Architectural Principle:** The system must distinguish among three separate questions: (1) Is PHP likely used? (2) Which application features exist? (3) Does available evidence demonstrate a security weakness? The system must never infer that a website is vulnerable merely because it uses PHP or exposes a feature.

1.1 Primary & Secondary Objectives

- **Primary:** Build a maintainable, testable, modular security platform that returns evidence-backed findings for an authorized target URL.
- **Secondary:** Provide a documented REST API, responsive dashboard, reproducible scan history, configurable rules engine, and multi-format exports (JSON, HTML, PDF).

2. INSTRUCTIONS FOR AI CODING AGENTS

All AI agents (Cursor, Claude Code, GitHub Copilot, etc.) contributing to this repository must follow these non-negotiable operational guidelines:

1. **Repository Inspection:** Inspect existing structures before generating files. Never duplicate existing features or create redundant utility scripts.
2. **Network Safety Boundary:** Strictly block all SSRF risks (loopback, private, link-local, multicast, and cloud metadata IPs). Account for DNS rebinding and redirects.
3. **Type Safety & Logging:** Enforce strict type annotations for all Python and TypeScript code. Use structured JSON logging; do not use raw `print()` statements.
4. **Test-Driven Completion:** No task is complete until unit and integration tests pass. Never fabricate test outputs or suppress security exceptions.
5. **No Destructive Exploitation:** Scans must remain non-destructive and passive/safe. Do not auto-submit forms, attempt brute-force, or exploit discovered vulnerabilities.

3. PROPOSED TECHNOLOGY STACK

LAYER	TECHNOLOGY	RESPONSIBILITY
Frontend	React + TypeScript + Tailwind CSS	User Interface, Real-time Dashboard, PDF/Report Viewer
Backend API	Python 3.11+ / FastAPI	API Routing, Pipeline Orchestration, Task Handling
HTTP Client	HTTPX (Async)	Controlled network fetching, strict rate limits & timeouts
Parsing / Processing	BeautifulSoup4 + Pydantic v2	HTML feature extraction, schema validation, serialization

LAYER	TECHNOLOGY	RESPONSIBILITY
Database & ORM	SQLite (Dev) / PostgreSQL (Prod) + SQLAlchemy 2.0	Persistent storage, relational mapping, transactional state
Migrations & PDF	Alembic / Jinja2 + WeasyPrint	Schema evolution, server-side HTML and PDF report rendering

4. REPOSITORY ARCHITECTURE

The codebase must enforce strict separation between API layers, domain logic, scanning engines, and frontend interfaces:

```
php-feature-scanner/
├── backend/
│   ├── app/
│   │   ├── api/          # Route handlers (health, scans, reports) & dependencies
│   │   ├── core/         # Config, security, SSRF validation, logging
│   │   ├── domain/       # Core models (Scan, Feature, Finding, Rules)
│   │   ├── scanner/      # Crawler, PHP detector, feature extractor, rule engine
│   │   │   └── rules/    # Individual passive security rules
│   │   ├── services/     # Orchestrators (scan execution, report generation)
│   │   ├── models/       # SQLAlchemy database schemas
│   │   └── reports/      # HTML/PDF Jinja2 templates & exporters
│   └── tests/            # Unit, integration, and security test suites
├── frontend/             # React SPA (components, hooks, pages, types)
└── docker-compose.yml    # Reproducible container deployment
```

5. SCAN PROCESSING PIPELINE

Scans must execute deterministically across 12 distinct pipeline steps:

1. **API Request Validation:** Validate URL syntax and user authorization.
2. **Network Scope Check:** Resolve DNS and verify IP against network restriction policies.
3. **Scan Record Initialization:** Persist scan with state `queued` and assign UUID.
4. **Base Fetching:** Retrieve root response via controlled HTTP client with strict limits.
5. **Fingerprint Analysis:** Run PHP detection engine against headers, cookies, and syntax.
6. **Bounded Crawl:** Discover same-origin links up to configured depth/page caps.
7. **Feature Extraction:** Parse forms, parameters, login screens, and file handling routes.
8. **Rule Execution:** Pass collected observations through the passive security rules engine.
9. **Finding Normalization:** Assign status (CONFIRMED , POTENTIAL , INCONCLUSIVE), severity, and confidence.
10. **Persistence:** Commit features, observations, and findings into storage.
11. **Report Generation:** Compile metadata into HTML, JSON, and PDF formats.
12. **State Finalization:** Transition scan status to `completed` or `failed`.

6. DOMAIN CONTRACTS & EVIDENCE SPECIFICATION

Findings must maintain a clear distinction between **Severity** (potential impact) and **Confidence** (evidence strength). Never conflate these two dimensions.

```
{
  "id": "7f8b2c4e-1a3d-4e5f-9b0a-8c7d6e5f4a3b",
  "scan_id": "3a1b2c3d-4e5f-6a7b-8c9d-0e1f2a3b4c5d",
  "rule_id": "PHP-SEC-HDR-001",
  "title": "Missing Strict-Transport-Security Header",
  "severity": "medium",
  "confidence": "high",
  "status": "potential",
  "affected_url": "https://authorized.example/login.php",
  "evidence": {
    "source": "http_response_header",
    "observation": "HSTS header absent in HTTPS response",
    "timestamp": "2026-10-29T08:30:00Z"
  },
  "remediation": "Configure 'Strict-Transport-Security: max-age=31536000; includeSubDomains' on web server."
}
```


7. FUNCTIONAL REQUIREMENTS SUMMARY

> **FR-002 Safety Requirement:** Network policy checks must execute *before* connection and *after* DNS resolution to prevent DNS rebinding attacks. All loopback (127.0.0.0/8, ::1), private (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), and cloud metadata endpoints (169.254.169.254) must be blocked.

- **FR-003 HTTP Communication:** Connection timeout 5s, response limit 5MB, max 3 redirects, TLS verification mandatory.
- **FR-004 PHP Fingerprinting:** Inspect `X-Powered-By`, `PHPSESSID` cookies, `.php` suffixes, and framework signatures. Return `likely`, `not_detected`, or `inconclusive`.
- **FR-006 Feature Discovery:** Classify forms (login, search, upload), query params, and file routes. Never submit form values automatically.
- **FR-007 Security Checks:** Passive rules covering headers, cookie attributes, version disclosures, and public configuration weaknesses.

8. REST API INTERFACE SPECIFICATION

METHOD	ENDPOINT	DESCRIPTION	EXPECTED OUTPUT
`GET`	`/health`	Health and readiness check	`{"status": "ok"}`
`POST`	`/api/v1/scans`	Initiate new scan for target URL	`{"id": "uuid", "status": "queued"}`
`GET`	`/api/v1/scans/{id}`	Retrieve scan status and summary	Scan lifecycle state & progress
`GET`	`/api/v1/scans/{id}/findings`	Get paginated findings with filters	List of structured findings
`GET`	`/api/v1/scans/{id}/report`	Export report (JSON, HTML, PDF)	Formatted document or stream

9. VERIFICATION & QUALITY GATES

Prior to merging or completing any implementation phase, AI agents must verify all quality gates:

- **Unit Testing:** 100% pass rate on URL validation, scope blocking, rule execution, and HTML parsing.
- **Integration Testing:** Verify complete pipeline execution against local mock test server fixtures.
- **Security Testing:** Validate SSRF bypass resistance, TLS enforcement, and proper output escaping in UI/PDF exports.
- **Static Analysis:** Formatting clean via `Ruff` (Python) and strict `tsc` checks (TypeScript).

10. DEFINITION OF DONE (RELEASE CHECKLIST)

The application is ready for initial release when all conditions below are verified:

COMPONENT	ACCEPTANCE CRITERIA	STATUS
Target Validation	Backend independently rejects invalid/prohibited URLs and SSRF targets.	VERIFIED
PHP Detection	Combines multi-indicator evidence; correctly handles missing signatures.	VERIFIED
Feature Spidering	Discovers bounded forms, upload boxes, and params without form submission.	VERIFIED
Security Engine	Generates structured findings with distinct severity, confidence, and status.	VERIFIED
Reporting	Renders consistent JSON, standalone HTML, and downloadable PDF reports.	VERIFIED
User Interface	React UI displays scan status, feature inventory, findings, and downloads.	VERIFIED

> **Deployment Note:** Docker containers must isolate the backend scanner worker with restricted network permissions, allowing outbound traffic only to validated target ports (80/443).